



Starburst: A Cost-aware Scheduler for Hybrid Cloud

Michael Luo, Siyuan Zhuang, Suryaprakash Vengadesan,
and Romil Bhardwaj, *UC Berkeley*; Justin Chang, *UC Santa Barbara*;
Eric Friedman, Scott Shenker, and Ion Stoica, *UC Berkeley*

<https://www.usenix.org/conference/atc24/presentation/luo>

This paper is included in the Proceedings of the
2024 USENIX Annual Technical Conference.

July 10–12, 2024 • Santa Clara, CA, USA

978-1-939133-41-0

Open access to the Proceedings of the
2024 USENIX Annual Technical Conference
is sponsored by





Starburst: A Cost-aware Scheduler for Hybrid Cloud

Michael Luo¹ Siyuan Zhuang¹ Suryaprakash Vengadesan¹ Romil Bhardwaj¹
Justin Chang² Eric Friedman¹ Scott Shenker¹ Ion Stoica¹

¹UC Berkeley ²UC Santa Barbara

Abstract

To efficiently tackle bursts in job demand, organizations employ hybrid cloud architectures to scale their batch workloads from their private clusters to public cloud. This requires transforming cluster schedulers into cloud-enabled versions to navigate the tradeoff between cloud costs and scheduler objectives such as job completion time (JCT). However, our analysis over production-level traces show that existing cloud-enabled schedulers incur inefficient cost-JCT trade-offs due to low cluster utilization.

We present Starburst, a system that maximizes cluster utilization to streamline the cost-JCT tradeoff. Starburst's scheduler dynamically controls jobs' waiting times to improve utilization—it assigns longer waits for large jobs to increase their chances of running on the cluster, and shorter waits to small jobs to increase their chances of running on the cloud. To offer configurability, Starburst provides system administrators a simple *waiting budget* framework to tune their position on the cost-JCT curve. A departure from traditional cluster schedulers, Starburst operates as a higher-level resource manager over a private cluster and dynamic cloud clusters. Simulations over production-level traces and real-world experiments on a 32-GPU private cluster show that Starburst can reduce cloud costs by up to 54-91% over existing cluster managers, while increasing average JCT by at most 5.8%.

1 Introduction

With the growing demand for workloads in artificial intelligence (AI) and data analytics, enterprises have built large, multi-tenant private clusters (henceforth, simply fixed clusters) coupled with high-end GPU accelerators [39, 40, 67]. These fixed clusters utilize cluster managers [46, 51, 71] that allocate resources to jobs based on a defined *scheduling policy*. These clusters also regularly experience large bursts in job volume during events such as upcoming deadlines or product launches, resulting in long queuing delays. Instead of investing in more on-premise hardware, many organizations are adopting hybrid cloud architectures [3, 7, 11, 16]. These architectures leverage the cloud to dynamically provision

additional resources during peak loads, offering a more cost-effective solution [1, 2, 8, 12, 14]. However, the clouds' illusion of an infinite resource capacity presents new challenges for *cloud-enabled* schedulers [28]. These schedulers must not only allocate jobs between a fixed cluster and the cloud, but also be aware of cloud costs. In this paper, we explore the problem of *when* to schedule jobs on the cloud to reduce cloud costs while avoiding long queuing delays.

When a cluster is full upon a job's arrival, the scheduler exercises two options: either wait or immediately send the job to the cloud. Existing cluster managers with cloud autoscaling opt for the latter—a strategy we term *No-Wait* [46, 51, 65, 71]. This approach eliminates queuing delays and reduces the average job completion times (JCT), but it introduces high cloud costs due to low cluster utilization. As illustrated in Fig. 1a, jobs A-D arrive in a burst at time $t = 0$. No-Wait assigns small 1-GPU jobs A and B to the cluster but immediately sends larger 2 and 4-GPU jobs C and D to the cloud, leaving idle resources after $t = 1$. No-Wait overlooks the option to wait, which delays job execution on the cluster to avoid provisioning expensive cloud resources.

If we choose to wait, the next question is: how long to wait? One simple policy, *Constant-Wait*, allocates a constant waiting time, $w(j) = C$, for all jobs, j , before they're sent to the cloud [17, 18]. In practice, selecting the right C is challenging. Short wait times reduce queuing delays but introduce high cloud costs and low cluster utilization, as illustrated in Fig. 1b ($C = 1$). Job C waits and executes at $t = 0.5$, but large 4-GPU job D, which is 8 hours long, fails to sufficiently wait to acquire 4 GPUs and is thus sent to the cloud. Conversely, long wait times reduce costs and are acceptable for longer jobs, but long waits block and unfairly delay short jobs from running.

These distinct issues highlight the tradeoff that cloud-enabled schedulers navigate: balancing cloud costs against job completion times (JCT), as depicted in Fig. 2. No-Wait immediately dispatches jobs if the cluster is full, leading to high cloud costs. Without the cloud, jobs can queue indefinitely, resulting in zero cloud costs and high JCT (No-Cloud, $C = \infty$) [31, 46]. Finally, *Constant-Wait* modulates

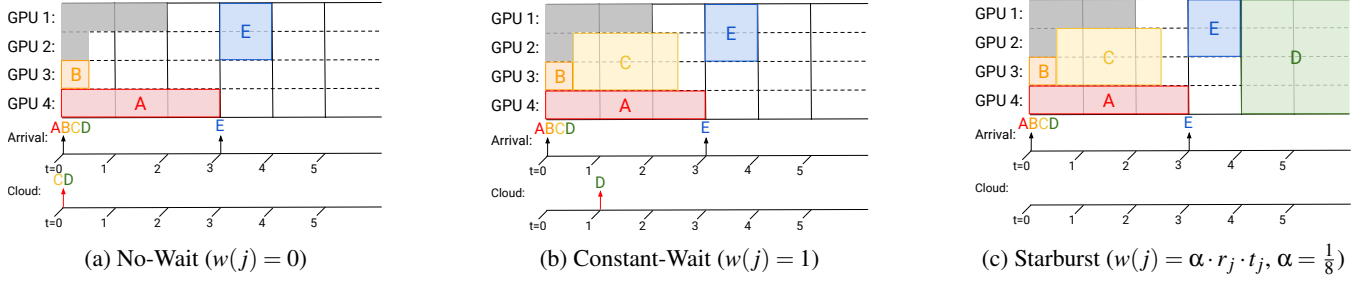


Figure 1: Gantt chart of job execution on a small 4 GPU cluster. Arrival of a job and its timeout to the cloud are depicted as arrows on the two time axes, with grey boxes indicating jobs running before $t=0$. (a) No-Wait under-utilizes the cluster, immediately sending jobs C and D to the cloud at $t = 0$. (b) With Constant-Wait ($w(j) = 1$), jobs wait a maximum of 1 timestep before moving to the cloud. Here, job C is scheduled on the cluster, while D is sent to the cloud. (c) Starburst dynamically allocates waiting time based on Compute-Wait, which allocates time proportional to a job’s resource requests (r_j) and runtime (t_j), and schedules jobs out of order. All jobs (A-E) run on the cluster.

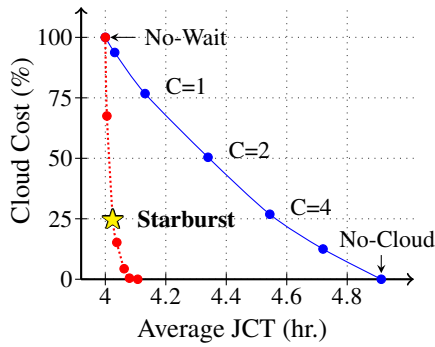


Figure 2: **Cost-JCT tradeoff for cloud-enabled schedulers.** No-Wait ($C=0$) incurs high cloud costs and low JCT, while No-Cloud ($C=\infty$) incurs no cloud cost but high JCT. Adjusting Constant-Wait’s parameter (C), the maximum wait time before jobs are sent to cloud, smoothly interpolates between these options and generates the right curve. Starburst (with Compute-Wait) shifts the Pareto curve downward toward the left curve and finds a suitable point via a *waiting budget*, indicated by the star icon.¹

cost and JCT based on varying values of C [17, 18].

We present Starburst, a general system that significantly improves cluster utilization to achieve a more efficient cost-JCT tradeoff. Our key insight is that waiting proportional to a job’s compute costs better balances cost and JCT; large expensive jobs, which better utilize the cluster, wait long enough to run on the cluster, while smaller jobs, with reduced costs, wait less and are more likely to run on the cloud.

As such, Starburst introduces a novel framework that dynamically assigns wait times based on jobs’ characteristics. We propose two scheduling algorithms: *Compute-Wait*, assuming knowledge of job runtimes, and *Star-Wait*, requiring no prior knowledge. *Compute-Wait* allocates a job’s maximum waiting time based on its total compute—the product of its predicted runtime and requested resources. Similar to *Compute-Wait*, *Star-Wait* is runtime and resource aware but assumes zero knowledge of job runtimes. To separate long-running jobs from short ones, it preempts cloud-running jobs that execute past a small time threshold

from cloud back to the cluster. Preempted jobs are then assigned wait times in proportion to their resource demands.

More importantly, assigning long waits to large jobs blocks smaller jobs further down in the queue. To reduce blocking, Starburst aggressively executes jobs *out-of-order* in the queue. This technique effectively schedules small jobs while large jobs wait for sufficient resources. Overall, Starburst is depicted in Fig. 1c, where jobs A-E all run on the cluster. Job E, which arrives later, executes out-of-order before job D. Job D under Compute-Wait is assigned a longer waiting time ($w(j) = \alpha \cdot 4 \cdot 8 = 4$ hours, where $\alpha = \frac{1}{8}$ is a hyperparameter) and thus executes at $t = 4$. Collectively, Starburst’s policies shift the cost-JCT curve towards a more Pareto efficient solution (Fig. 2), enabling Starburst to achieve up to 99% cluster utilization and come within 5% of the optimal solution (§6.5).

Given diverse workloads and budgetary constraints across organizations, there’s no one-size-fits-all position on the cost-JCT tradeoff. Starburst offers a simple *waiting budget* feature that exposes a singular knob for system admins to determine their position on the cost-JCT curve. This is defined as a maximum percentage, $P\%$, that administrators are willing to accept as an increase in the average job runtime. A departure from cluster schedulers, Starburst is a general higher-level resource manager that mediates jobs *across clusters*, over a private cluster and dynamically provisioned cloud clusters, and is transparent to its users, requiring no modifications to the jobs. We also implement a high-fidelity simulator for testing Starburst on multiple production traces [39, 40]. Our results on a 32 GPU physical cluster show that Starburst decreases costs by 80% while increasing JCT by only 5.8% over No-Wait. For large-scale simulations, Starburst decreases costs by 54-91% while increasing JCT by at most 5%. Finally, Starburst is robust to data transfer overheads, highly bursty workloads, and lower-level queueing and bin-packing policies.

The primary contributions of this paper are:

- We formalize the cloud cost-JCT tradeoff as a scheduling

¹Out-of-order scheduling improves JCT in the no cloud setting but creates a long-tail in JCT. In the hybrid cloud setting, since jobs timeout to cloud, we do not observe unfair job delays in Starburst (§6.4).

problem in the hybrid cloud setting. (§2)

- Starburst dynamically assigns wait time based on jobs’ characteristics to maximize cluster utilization, better balancing cost and JCT. Starburst can leverage knowledge of job runtimes but can also operate without prior knowledge. (§4)
- Starburst’s *waiting budget* framework exposes a single knob for system administrators to define their position on the cost-JCT curve. (§4)
- Our system is readily deployable and demonstrates significant cost-performance improvements. (§5,6)

2 Background

We provide an overview of cloud-enabled schedulers and their corresponding policies, objectives, and metrics, which details relevant context for Starburst. Starburst’s key assumptions, such as preemption and data locality, are described in the main section. (§4)

Policies. In the hybrid cloud setting, schedulers must consider a broader set of scheduling policies compared to traditional cluster schedulers. Typically, cluster schedulers determine job placement on a fixed cluster based on two types of scheduling policies, composed of queueing policies, such as First Come First Serve (FCFS) or Shortest Job First (SJF), and bin-packing strategies, such as Best-Fit and First-Fit [46,51,71].

In contrast, *cloud-enabled* schedulers distribute jobs between the fixed cluster and dynamically provisioned cloud resources. Hence, a cloud-enabled scheduler not only manages scheduling policies over a fixed cluster, but also *waiting policies*, which determine when jobs move to cloud if cluster resources are unavailable. A simple waiting policy is *Constant-Wait*, where all jobs j wait for at most $w(j) = C$ hours before jobs are sent to the cloud [17, 18]. This is equivalent to queueing theory with reneging, where impatient customers leave the system after waiting too long [21, 35]. Another proposed waiting policy is *No-Short-Jobs* (*No-SJ*), which immediately sends all short jobs to the cloud and serves as one of our baselines in evaluation [18].

Objectives. Cloud-enabled schedulers must balance multiple objectives, such as cloud costs and traditional cluster scheduling goals, including job completion time (JCT), throughput, and fairness. Notably, cloud costs conflict with traditional objectives, thereby forming Pareto curves. For instance, Fig. 2 illustrates the tradeoff curve between cost and JCT, where Constant-Wait traces out various points on the tradeoff by adjusting the timeout threshold, C .

We model the cost-JCT tradeoff as a constrained optimization problem, minimizing cloud costs while constraining JCT, or equivalently, jobs’ total waiting times. Consider a set of batch jobs j , each requesting r_j resources and running for t_j time, the cloud cost function $C(\cdot)$, and the optimization variables w_j , which measures waiting times for job j , and

c_j , a 1/0 indicator variable indicating whether a job runs on cluster or cloud respectively. Our simplified cost-JCT formulation is defined as follows:

$$\begin{aligned} \min_{c_j, w_j} \quad & \sum_j (1 - c_j) \cdot C(j) \approx \max_{c_j, w_j} \sum_j c_j \cdot r_j \cdot t_j \\ \text{s.t.} \quad & \sum_j w_j \leq B \\ & F(j) = 1 - c_j, \forall j \end{aligned}$$

, where B is the *waiting budget*, a user-defined threshold for the total waiting time across all jobs (further detailed in § 4.3), and $F(\cdot)$ is a binary function that determines if a job cannot fit on the cluster. For brevity, the cluster’s bin-packing constraints are abstracted away as part of $F(\cdot)$; the complete formulation for cloud-enabled schedulers is presented in § A.2. Starburst extends this formulation by assuming that cloud costs, $C(\cdot)$, are proportional to a job’s compute, the product of its resources and time. This assumption aligns with common cloud pricing models, which typically charge based on compute usage. Consequently, maximizing cluster utilization minimizes cloud costs, as jobs that start on the cluster are completed there.

Metrics. This work evaluates cloud cost and JCT across a wide range of *system loads*, which measure the ratio of arriving job input load over the cluster’s compute capacity. Mathematically, for a set of jobs j , each requesting r_j resources and running for t_j time, arriving over a fixed period, T , and a private cluster with R total resources, system load is defined as $s = \sum_j r_j \cdot t_j / (RT)$.

Importantly, we evaluate scenarios where system load exceeds the cluster’s capacity ($s \geq 1$). This is motivated for two reasons: First, the cloud offers the illusion of “infinite” resources to accommodate arbitrarily high system loads, and second, real-world workloads exhibit bursty behavior, where incoming job volume often exceeds the fixed cluster’s capacity. Note that traditional cluster schedulers without cloud struggle in this setting; classic queueing theory shows that JCT exponentially increases as system load approaches cluster capacity ($s \approx 1$) [31]. Most importantly, we observed cluster managers with cloud autoscalers [10, 46], which implicitly assume No-Wait, scaled too aggressively and failed to sufficiently utilize the base cluster (85%) when the system load is twice the cluster’s capacity ($s = 2$). Low cluster utilization during high system loads serves as our motivation for developing Starburst.

3 Motivation

Starburst’s goal is to maximize cluster utilization, which minimizes the cost of jobs sent to the cloud. This challenge goes beyond just packing jobs on the cluster—it also involves correctly *choosing* which jobs should run on the cluster. When an incoming job is unschedulable due to insufficient cluster resources, cloud-enabled schedulers choose between

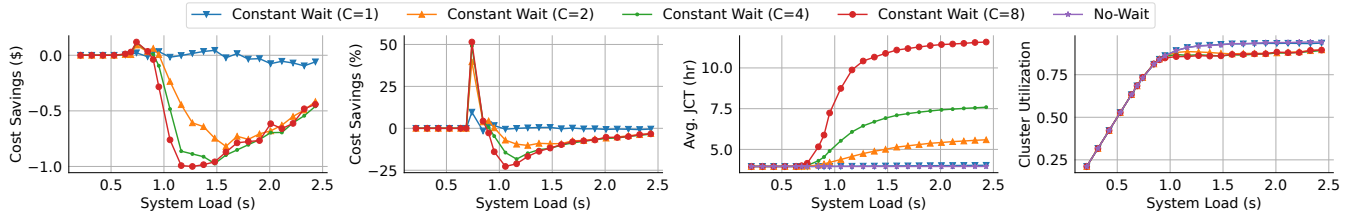


Figure 3: **Absolute (\$, normalized) and Relative Cost Savings (%) compared to No-Wait, JCT (hr.), and Cluster Utilization (%) for Constant Wait ($C = 1, 2, 4, 8$) over Microsoft Philly production trace [40].** Positive cost savings indicate lower cloud costs than No-Wait. Small waits ($C = 1$) do not significantly impact cost savings and JCT. Long waits ($C = 8$) yield better savings at low system loads ($s < 1$) but increase costs at high loads ($s \geq 1$); however, under high system loads, absolute cost savings remain a minor percentage of No-Wait’s total costs due to frequent cloud spillover and large cloud costs.

two options: either *wait* for additional cluster resources or immediately *send* the job to the cloud (No-Wait). No-Wait offers the advantage of immediate job execution, but incurs high cloud costs. To illustrate, if a large job arrives and the cluster is full, No-Wait promptly moves the job to the cloud. This overlooks the potential benefits of delaying job execution, which allow the job to run on the cluster if resources eventually become available.

Seeing the potential advantages of waiting, we turn our attention to the question: how long to wait? Our analysis over production-level traces show that the answer is nuanced (§3.1); it introduces two challenges—job starvation and head-of-line blocking (§3.2). The scheduler must balance between allocating many small jobs on the cluster, which risks sending large expensive jobs to the cloud, and waiting for sufficient resources to schedule large jobs, which blocks small jobs from executing. These problems are both addressed in the design of Starburst’s scheduling policies (§4).

3.1 Short and Long Waits

We ablate the effects of short and long waits using a widely-used trace over Microsoft’s internal Philly cluster [40]. This trace is representative of other batch workloads and is further characterized in our evaluation (§6). Our findings reveal that short waits offer negligible cost savings compared to No-Wait. Long waits, however, function as a double-edged sword: they can substantially reduce cloud costs under low system load but also risk increasing costs under high loads.

We begin with a simple waiting policy, Constant-Wait, that assigns all jobs a timeout of C before they’re sent to the cloud. Fig. 3 evaluates cost savings (over No-Wait), JCT, and cluster utilization ablated over short and long waits. When compared to No-Wait, short waits ($C = 1$) minimally impact cloud costs, with cost savings ranging between $-1 - 10\%$. On the other hand, long waits ($C = 8$) are more nuanced. For system loads below cluster capacity ($s < 1$), longer waits saves costs by up to 52%. However, for system loads above cluster capacity $s \geq 1$, Constant-Wait incurs a 25% increase in cloud costs over No-Wait. Correspondingly, cluster utilization also drops from 91%, as seen in No-Wait, to 85%.

Paper	(%)	GPU Count			
		[0, 1)	[1,8)	[8,32)	[32, ∞)
Philly [40]	Count	0.01%	97.5%	2.4%	0.01%
	Cost	0%	35.3%	62.7%	2.0%
Helios [39]	Count	61.3%	25.9%	11.6%	1.2%
	Cost	0.08%	10.8%	47.4%	41.7%
Alibaba [67]	Count	37.7%	45.7%	5.5%	11.1%
	Cost	0.07%	0.44%	1.1%	98.4%

Table 1: **Workload and Cost distribution over public cluster traces.** Small jobs occupy a majority of jobs, while large jobs occupy a majority of costs.

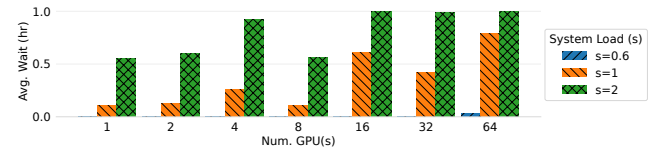


Figure 4: **Job waiting times vs GPU(s) requested across varying levels of traffic for Constant-Wait ($C=1$).** Average waiting time increases as the GPU resource request increases. When system load exceeds cluster capacity ($s = 2$), all 16+ GPU jobs are sent to the cloud.

3.2 Problems with Waiting

3.2.1 Job Starvation

The first problem, *Job Starvation*, occurs when jobs are assigned insufficient waiting times (i.e., small values of C). The problem is that large jobs fail to acquire sufficient resources in time and are consequently sent to the cloud. Here, large jobs are essentially *starved* from running on the cluster. This behaviour leads to low cluster utilization, since large jobs account for a majority of compute costs. In Tab. 1, our analysis across multiple traces show that large jobs above 8 GPUs account for 65-99% of total costs, despite small jobs accounting for over 80% of total jobs. As a result, smaller jobs are repeatedly scheduled on the cluster. Due to the high turnover rate of small jobs, we observed frequent, brief periods of idle resources in our execution traces (§A.4).

To quantify the impact of starvation, Fig. 4 plots average job waiting time as a function of requested GPU(s) for Constant-Wait ($C = 1$). The chart reveals large jobs face longer waiting times than small jobs. For instance, when system load is at cluster capacity ($s = 1$), jobs requesting 64 GPUs wait 8 times longer than 4 GPU jobs, thereby indicating a higher likelihood of timing out and being sent to the cloud. Notably, when system load exceeds cluster capacity ($s = 2$), all jobs requesting 16+ GPUs run on the cloud, as the average waiting time is the same as the timeout threshold, $C = 1$.

3.2.2 Head-of-line Blocking

The second issue, *head-of-line (HoL) Blocking*, arises when jobs are assigned long waiting times (i.e. large values of C) and system load exceeds cluster capacity ($s \geq 1$). In this situation, scheduling a large job at the front of the queue blocks subsequent, smaller jobs from being scheduled.² This results in idle resources, as the scheduler awaits for sufficient resources to become available to schedule the large job. This issue worsens as waiting times grow longer. Our execution traces show large and long periods of cluster under-utilization due to HoL blocking (§ A.4).

HoL blocking presents an opportunity cost for the scheduler: should it wait for the large job to execute or proceed with smaller jobs in the queue? As system load rises, the opportunity cost increasingly favors scheduling multiple smaller jobs. Higher system load leads to longer job queues, which presents more opportunities to pack smaller jobs and improve cluster utilization. Fig. 5 illustrates the impact of head-of-line blocking on waiting times, $C \in \{0, 1, 2, 4\}$, and system loads, $s = 0.8, 1.5$, over our execution traces. We iterate through all periods of under-utilization in our traces and classify them as either idle space (no jobs in queue) or HoL space (at least one job in queue, due to HoL blocking). Fig. 5a, which corresponds to low system load ($s = 0.8$), shows that longer waiting times increases HoL space and lowers cloud costs. Under high system load, Fig. 5b reveals that longer waiting times significantly increases HoL space (+80%), leading to higher cluster under-utilization and cloud costs.

4 Starburst Policy

Starburst introduces simple heuristics to address job starvation and head-of-line blocking respectively. Combined, they significantly increase cluster utilization and shift the cost-JCT curve towards a more efficient solution (§4.1,4.2). Then, we discuss how system admins can tune their desired points on the cost-JCT curve using the *waiting budget* framework (§4.3). Finally, we propose two simple methods to reduce JCT without impacting costs (§4.4). Starburst’s core algorithm is shown in Alg. 1.

²Many cluster managers assume HoL blocking to schedule large jobs, including exclusively reserving a separate cluster for only large jobs [40,67], assigning high priority to large jobs [32,46], or allowing large jobs to block [39].

Algorithm 1 Starburst Pseudocode

```

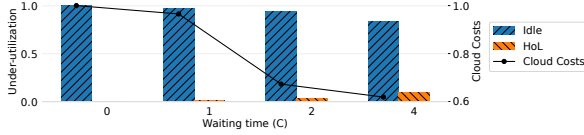
1: When async event triggers scheduler
2: if job  $j$  arrives then
3:   Update hyperparameter ( $\alpha, \beta$ ) for  $w^*(j)$  (§4.3)
4:   Set  $j.timeout$  to  $w^*(j)$  (§4.1, §4.4)
5:   Add  $j$  to queue
6: end if
7: if job  $j$  times out then
8:   Remove  $j$  from queue
9:   Execute job on cloud
10: end if
11: if cluster has available resources then
12:   for job  $j$  in queue do
13:     if  $j$  can fit in cluster then
14:       Remove  $j$  from queue
15:       Execute job on cluster
16:     else
17:       continue (§4.2)
18:     end if
19:   end for
20: end if
21: if job  $j$  runs past  $T_{max}$  on cloud then
22:   if  $j.preempted = \text{false}$  then ▷ Star-Wait (§4.1)
23:     Set  $j.preempted = \text{true}$ 
24:     Add  $j$  to queue
25:     Terminate job  $j$  on cloud
26:   end if
27: end if

```

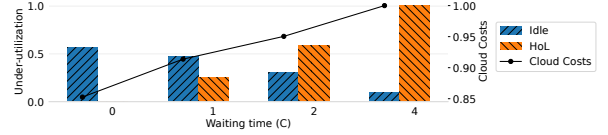
Assumptions. Starburst assumes submitted jobs are batch jobs that run to completion. Examples include AI training and fine-tuning tasks over popular models (e.g. ImageNet [45], BERT [25]), data analytics (e.g. Spark [73], MapReduce [24]), and other HPC workloads (e.g. [49]). Second, Starburst assumes that progress is lost upon job preemption. However, our work also includes checkpointable jobs with a simple modification: Starburst approximates a checkpointable job with a sequence of smaller non-checkpointable jobs, where each job represents the time period between two consecutive checkpoints. Finally, we assume job runtimes are similar for both cluster and cloud environments; this simplifies the model by eschewing the benefits of data locality on cluster and diverse hardware in the cloud, which are beyond the scope of our study. However, we show that Starburst is resilient to data transfer overheads in §6.6. Our real system, which assumes that job data is stored either on a cluster or in a cloud bucket, accesses data via FUSE-based software [9] or direct download, regardless of the execution location.

4.1 Dynamic Waiting

Reducing job starvation requires increasing wait times for large jobs, so that large jobs can acquire sufficient resources on the cluster. Starburst’s waiting policies dynamically



(a) System load near cluster capacity ($s = 0.8$)



(b) System load above cluster capacity ($s = 1.5$)

Figure 5: **Comparison of cluster under-utilization types over varying system loads based on the Philly trace [40].** The impact of head-of-line blocking as a cause for cluster under-utilization grows significantly as both system load and waiting time increase.

aim to assign jobs' waiting times in proportion to their computation—a product of a jobs' resource requests and runtime. These policies apply across various workloads and resource types, although we focus on GPUs and CPUs.

Compute Wait. If job runtimes are known, computing wait times is simple. For *Compute-Wait*, if job j requests a set of resource types $r_{j1}, r_{j2}, \dots, r_{jn}$ (e.g., GPUs, CPUs, disk, memory) and runs for t_j hours, job j waits a maximum of:

$$w^*(j) = \alpha \cdot \left(\sum_{k=1}^n W_k \cdot r_{jk} \right) \cdot t_j \quad (1)$$

hours before a job times out to the cloud. Here, W_k is the relative cost for resource type k , and α is a hyperparameter controlled by the waiting budget, further detailed in §4.3. Our experiments consider only GPU and CPU resource types (r_1, r_2), with $W_2 : W_1$ set to 1/50, the cost ratio of vCPU to V100 in Tab. 5, which evaluates the absolute cloud costs of different resource types.

Compute-Wait increases the likelihood of executing large jobs on the cluster. First, larger and longer jobs are assigned long waiting times, granting them an extended window to acquire necessary resources. Second, Compute-Wait allocates smaller and shorter jobs short waiting times. This ensures a higher proportion of small jobs timeout to the cloud, which effectively reduces the overall queue length and accelerates the scheduling of large jobs.

Without time estimates. *Star-Wait*, a pragmatic approximation for *Compute-Wait*, assumes no knowledge of job runtimes. *Star-Wait* relies on a *preemption* mechanism that terminates jobs on the cloud if a job runs for more than T_{max} time and migrates them back to the cluster. This ensures that short jobs finish before T_{max} while longer jobs are given a second chance to wait for cluster resources. Our experiments set T_{max} to be less than one-eighth of the average job runtime. *Star-Wait*'s preemption is detailed at the end of Alg. 1.

Star-Wait follows a piecewise function for its waiting policy. For job j that requests a set of resource types, $r_{j1}, r_{j2}, \dots, r_{jn}$ (e.g., GPUs, CPUs, disk, memory), job j waits for a maximum of:

$$w^*(j) = \begin{cases} 0 & \text{if } j.preempted = \text{false} \\ \beta \cdot \left(\sum_{k=1}^n W_k \cdot r_{jk} \right) & \text{if } j.preempted = \text{true} \end{cases} \quad (2)$$

hours, where W_k is the relative cost of resource type k and β is a hyperparameter. Initially, *Star-Wait* applies a No-Wait

policy, which immediately sends jobs to the cloud if the cluster is full. If a job on the cloud runs more than T_{max} , it is preempted from the cloud and reassigned to the cluster under a Resource-Wait policy, which assigns wait times proportional to the job's requested resources. If a preempted job is still unable to execute on the cluster, it reruns on the cloud without interruption. This policy mimics Compute-Wait as small jobs are effectively assigned zero wait while long jobs are assigned resource-proportional wait time. Although *Star-Wait* performs slightly worse than Compute-Wait, *Star-Wait*'s preemption-based model offers a practical out-of-the-box solution.

4.2 Out-of-Order Scheduling

Compute-proportional waiting allocates more wait to larger and longer jobs, but this incurs significant under-utilization due to head-of-line (HoL) blocking. We propose *Out-of-Order* (OO) scheduling, achieved by simply looping through the job queue and greedily scheduling any job that can fit whenever the cluster is available. If a job can't fit on the cluster, *Starburst* skips the job and moves on to the next job in the queue. This behavior is detailed in Alg. 1, where *Starburst* iterates through the queue despite encountering unschedulable jobs. Consequently, *Starburst* is more likely to schedule smaller jobs requiring fewer resources than the large head-of-line job.

Out-of-Order scheduling is particularly effective with waiting policies, where jobs timeout and leave to the cloud. First, there are less jobs in the system, increasing the likelihood that large jobs execute on the cluster. In traditional scheduling, jobs wait forever since there is no cloud, so OO scheduling repeatedly assigns many small jobs to the cluster, which may indefinitely delay the execution of larger jobs. Second, Out-of-Order scheduling complements *Starburst*'s Compute-Wait policy, as large jobs are allocated longer waiting times and can tolerate longer delays from scheduling small jobs.

Finally, *Starburst*'s behavior closely matches backfill scheduling [48, 51, 71], a complex scheduling technique which requires precise time estimates and job reservations. Compute-Wait implicitly *reserves* large jobs in the future while Out-of-Order scheduling *packs* smaller jobs between reservations. We evaluate backfill scheduling vs *Starburst* in §A.7.

4.3 Waiting Budget

This section presents the *waiting budget* framework, a mechanism that provides administrators a simple way to align system configurations with their organization's budget

constraints and workload characteristics.

Waiting budget, B defines the maximum possible waiting time for all jobs (which can only be achieved if all jobs timeout). This implicitly upper bounds average JCT, since JCT is a sum of average runtime plus waiting time. The system administrator specifies a desired percentage increase, $P\%$, over average job runtime, T , for N jobs, which sets the waiting budget as $B = \frac{PTN}{100}$. The waiting budget determines the waiting policy's hyperparameter. We illustrate this over two examples, Constant-Wait and Starburst's Compute-Wait. For Constant-Wait ($w(j) = C$), the budget is evenly divided over all jobs: $C = \frac{B}{N} = \frac{PT}{100}$. For Compute-Wait, assuming job size and runtime are independent, α depends solely on P and the average job size $\mathbb{E}[r]$: $\alpha = \frac{B}{\sum_j r_j t_j} \approx \frac{PTN}{N \cdot \mathbb{E}[r] \cdot T \cdot 100} = \frac{P}{100 \cdot \mathbb{E}[r]}$. The independence assumption holds for our workloads, where the correlation between job size and runtime is small ($r \leq 0.15$). In both cases, Starburst only requires knowledge of the workload's average runtime or size to determine the waiting policy's hyperparameter. Similarly, Star-Wait's hyperparameter (β) depends on the same statistics (§A.3). Consequently, Starburst continually updates workload statistics as more jobs are submitted.

4.4 Immediate Job Offloading

Some jobs can be immediately *offloaded* to the cloud upon arrival. Such jobs are marked with negative waiting time ($w^*(j) = -1$). This design mimics *balking* in queueing theory, where customers preemptively leave the system upon predicting long queueing delays. In a real-world setting, deriving an accurate prediction of job waiting times is computationally intensive [18]. As such, this section identifies simple heuristics that Starburst can exploit to immediately offload specific jobs, without significantly impacting cost.

CPU-only Jobs For our workloads, Starburst immediately sends CPU-only jobs to the cloud for two main reasons. First, GPU prices are at least an order of magnitude greater than that of vCPU prices. For example, older GPUs such as V100 (\$2.50/hr) cost 50x more than 1 vCPU (\$0.05/hr) [42, 43, 70]. Second, scheduling CPU jobs can also cause resource stranding, where active CPU jobs can prevent incoming GPU jobs from being scheduled. This problem arises frequently in the Helios trace (Tab. 1), where CPU jobs account for 60% of jobs yet only occupy 0.08% of costs [39].

Limiting Queue Size Starburst upper bounds the length of the job queue, denoted as $Q_{\max}$. If queue length surpasses $Q_{\max}$, jobs are immediately sent to the cloud. As long as $Q_{\max}$ is not small, this strategy barely impacts costs. It provides theoretical asymptotic guarantees; JCT converges to average job runtime as system load grows arbitrarily large ($s \rightarrow \infty$) [66].

5 Starburst Design

Starburst is designed as a higher-level resource manager that redirects jobs between a cluster manager and the cloud. Figure 6 presents Starburst's architecture and the overall job flow.

5.1 Architecture

Starburst is a hybrid resource manager that can express complex policies and manages jobs between two cluster types, the fixed private cluster and dynamic cloud clusters. Cloud clusters are composed of independently provisioned cloud instances across different cloud vendors and regions. We choose this design since existing cluster autoscalers are monolithic, lack exposable customization to modify autoscaling policies, and are limited to single regions only, which limits resource availability [46, 71]. Starburst's components are classified into three categories:

Schedulers: Starburst's control loop in Alg. 1 is split across the hybrid scheduler and scheduler plugin on the cluster manager. The hybrid scheduler controls scheduling jobs across clusters while the scheduler plugin controls scheduling jobs within a fixed cluster.

1. Hybrid Scheduler. The hybrid scheduler schedules jobs to either the cluster or the cloud. It is triggered by user-submitted or evicted jobs. During job submission, the scheduler calculates the job's timeout threshold with a waiting policy (e.g. Compute-Wait). This versatile design allows for arbitrary job timeouts. Importantly, the hybrid scheduler implements its own internal queuing policies and thus performs *Out-of-Order* scheduling; hence, the underlying cluster manager only considers bin-packing policies.

2. Cluster Manager. The cluster manager is responsible for scheduling jobs to nodes resources. The cluster manager's scheduler plugin modifies the default cluster manager's scheduling behavior and implements its own scheduling policies, such as best-fit binpacking. Note that the cluster manager does not evict or preempt jobs.

Rescheduler. The rescheduler determines the criteria under which a job is terminated and rerouted to the hybrid scheduler.

1. Monitor. The monitor continuously observes jobs over the cluster manager and cloud, providing real-time updates on their status and various job statistics.

2. Evictor. The evictor checks with the monitor for two cases: 1) pending jobs that have exceeded their assigned timeouts, or 2) long-running cloud jobs that have not been preempted before (Star-Wait). The evictor terminates such jobs and resubmits them to the hybrid scheduler.

Cloud Orchestrator: The cloud orchestrator is tasked with managing job execution in the cloud. To reduce costs, it assumes one job per set of cloud instances and terminates such instances upon job completion or preemption.

1. Translator. The translator bridges the cluster manager's job interface with the cloud executor. Users provide job specifications for the cluster manager, which the translator translates for the Executor to process and run on the cloud.

2. Provisioner. The provisioner assesses a job's resource requests and provisions the most suitable cloud resource that closely matches the job's needs.

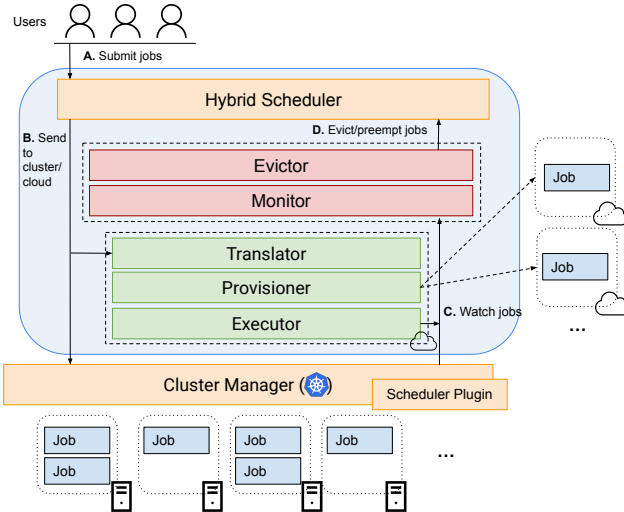


Figure 6: Starburst’s system architecture and job lifecycle. Hybrid scheduler and cluster manager orchestrate jobs across clusters and nodes. Monitor and evictor tracks and reschedules jobs. Translator, provisioner, and executor manage job execution on the cloud.

3. Executor. Once resources are provisioned, an executor oversees the actual job execution on the designated VM.

5.2 Implementation

Starburst is implemented in both simulation and as a real system, totalling 5k lines of code in Python. We delineate implementation details for simulator and real-life implementation.

Simulation. The simulator is a continuous-time, discrete event model activated by a primary control loop, with pseudo code in Alg. 1. The control loop triggers when a job is added, timed out, preempted, or completes on the fixed-cluster. The simulation offers various configuration parameters, such as waiting policies, waiting budgets, configurations over multiple job traces, arrival rates, and fixed cluster sizes. Finally, the simulator’s fidelity mirrors the real system, verified in §6.2.

Real-life Implementation. Starburst is implemented as a manager over a local Kubernetes cluster [46] and cloud clusters, provisioned by Skypilot [70] over different clouds and regions. Users can submit a Kubernetes job directly to the hybrid scheduler without any modifications. The hybrid scheduler employs an asynchronous, event-driven architecture, where a `JobAddEvent` is generated when either a user submits Kubernetes jobs or the Eviction module resubmits jobs. The hybrid scheduler invokes the `WaitPolicy` class to determine the job’s timeout threshold and subsequently adds timeout value and cluster as Kubernetes annotations. Finally, the hybrid scheduler tracks the state of all pending jobs via an internal queue and implements *Out-of-Order* scheduling.

We developed a custom scheduler plugin, *Chakra*, using the Kubernetes (K8s) Python API, which is entirely optional. Although K8s’ default load-balancing pod-placement (worst-fit) is optimized for services, it is sub-optimal for batch jobs. Hence, *Chakra* also implements best-fit binpacking. When

the hybrid scheduler sends jobs to the cluster, K8s identifies *Chakra* as the intended scheduler, thus permitting *Chakra* to intercept job submission and perform cluster binpacking, binding pods to nodes.

Pending Kubernetes and Skypilot jobs are tracked by the monitor, akin to an informer that maintains an up-to-date cache of jobs. Given that Kubernetes Python API offers only watching, an asynchronous monitor for events, we develop a custom `Informer` class in Python. The evictor’s control loop regularly scans the `Informer`’s cache to reschedule jobs based on job state. Rescheduled jobs are annotated with either `ToCloud` or `ToCluster` if they originate from the cluster or cloud respectively. Rescheduled cloud jobs are also marked with a `Preempted` annotation.

Our hybrid scheduler interfaces with the Skypilot API for jobs with the `ToCloud` annotation. The Kubernetes job YAML is seamlessly converted to a Skypilot API through the dedicated translator component. Skypilot manages both provisioner and executor components. This integration enables Starburst to select the most cost-effective instances across various regions and clouds and increases resource availability. Job submissions are orchestrated using Ray’s job submission API [54] running on Skypilot VMs. We implement an *autodown* feature, which terminates VMs when jobs complete.

6 Evaluation

We evaluate Starburst’s performance and find that:

- Our real-world experiments and simulations demonstrate that Starburst significantly reduces cloud costs by 54-91% and reaches 99.2% cluster utilization, while increasing JCT by merely 5%. (§6.2, 6.3)
- Starburst with **no time estimator** (No-TE) attains similar performance to Starburst (within 16% costs). (§6.2, 6.3)
- Starburst’s policies significantly shift the cost-JCT curve downwards. It provides a more flexible approach than Constant-Wait for adjusting the waiting budget. (§6.4).
- Remarkably, Starburst’s performance approaches the optimal MILP solution, deviating by merely 5%. (§6.5)
- Starburst exhibits resilience to data gravity, highly bursty workloads—doubling cost savings—and various queueing and binpacking policies. (§6.6).

6.1 Experimental Setup

Cluster Testbed. Our testbed private cluster consists of 4 nodes, totalling 32 NVIDIA V100 GPUs and 384 vCPUs. Cloud VMs are dynamically provisioned by Skypilot over GCP instances across regions and zones. The Starburst scheduler runs on a separate GCP instance.

Metrics. We focus on cost savings, average JCT, and cluster utilization for our key metrics. Since No-Wait incurs the highest cloud costs on the cost-JCT pareto curve (see Fig. 2), we report *normalized* absolute (\$) and relative (%) cost savings over No-Wait. These metrics are evaluated over system load (s), the ratio of job input load to cluster capacity.

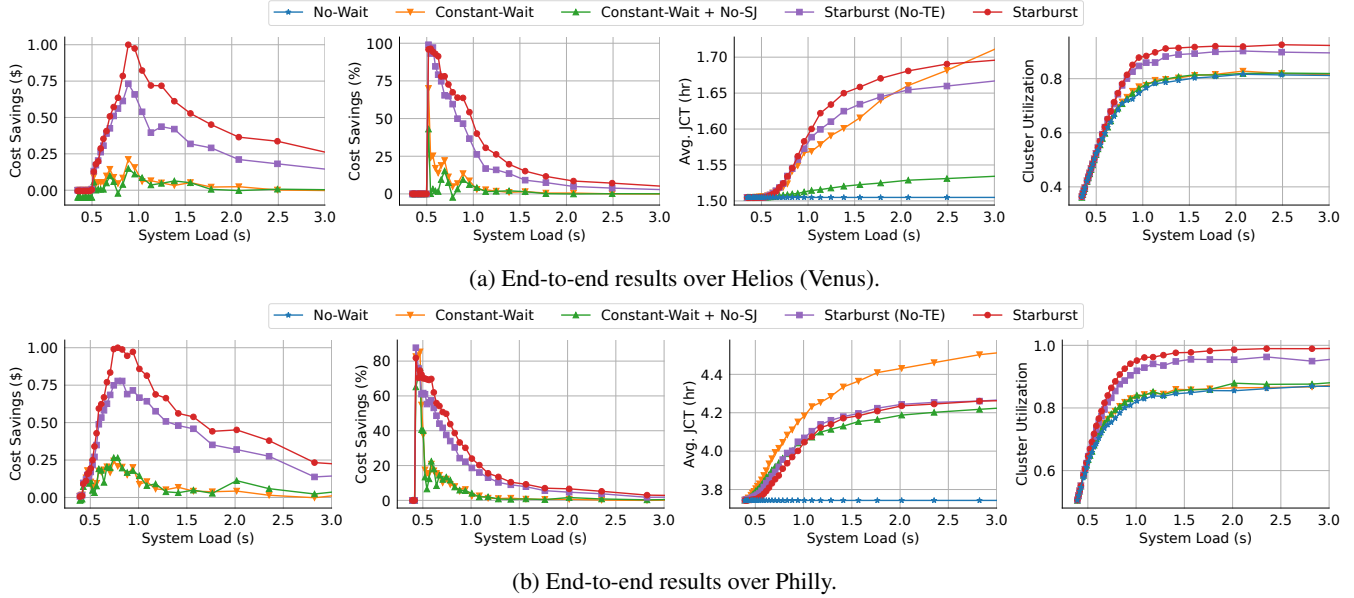


Figure 7: **Cost savings, JCT, and cluster utilization for end-to-end simulation.** Starburst substantially outperforms Constant-Wait for cost savings while achieving similar, if not smaller, average JCT across various system loads. The primary advantage of Starburst comes from its ability to increase cluster utilization, offering improvements of up to 15% over baselines.

Task	Model	Dataset	# GPUs
CV	EfficientNetV2 [63]	CIFAR-10 [44]	1, 2, 4, 8
	MobileNetV3 [38]		
	ResNet-50 [36]		
	ResNet-101 [36]		
	ResNext-50 [69]		
	VGG-11 [61]		
NLP	BERT [25] (tiny, mini, small)	WikiText-2 [27]	4, 8
	GPT-2 [58] (tiny, mini, small)	WikiText-103 [26]	
	—	WMT-16 (fi-en) [60]	

Table 2: **Models used for real-life experiments.** We sample training jobs from a cartesian product of the configurations above.

Baselines. We assess two variants of Starburst: one with a time estimator (Compute-Wait) and with **no** time estimator (Star-Wait, or No-TE), both using Out-of-Order scheduling. Starburst is evaluated against No-Wait ($w(j) = 0$), Constant-Wait ($w(j) = C$), and the **state-of-the-art** Constant-Wait + No-SJ (Short Jobs) [17, 18], which assumes time estimates and immediately sends short jobs under 15 minutes to the cloud. Similarly, Starburst (No-TE) preempts jobs exceeding 15 minutes in the cloud back to the cluster. To fairly compare between Starburst and other baselines, a waiting budget, $B = (P/100) \cdot N \cdot T$, defined in §4.3, is distributed among all N jobs. For all our experiments, we set $P = 25$; however, this value can be adjusted by the admin. We evaluate the budget’s sensitivity in §6.4.

Real-life Workload. Our real-life experiments evaluate Starburst over AI fine-tuning jobs. Our batch jobs, shown

in Tab. 2, vary across tasks, modalities (CV, NLP), models, datasets, GPU counts, and training epochs—which influences job runtimes from several minutes to five hours. This time also includes downloading datasets remotely. For our workload trace generation, we first synthesize a virtual trace by sampling job arrivals and runtimes from a Poisson process λ and exponential distribution $1/\mu$, with average runtime of 45 minutes. We then sample GPU count for jobs’ resource requests from a categorical distribution that aligns with the Microsoft Philly trace (see Tab. 1) [40]. Finally, for each job in the virtual trace, we match the actual training job that aligns most closely with the virtual job’s resource request and runtime.

Simulator Traces. Starburst’s simulator experiments rely on two publicly available production traces, Philly [40] and Helios [39]. The Microsoft Philly trace consists of $\sim 100K$ jobs submitted over the course of 3 months and considers only one resource type, GPU. It features gang-scheduled jobs, with an average job duration of 4 hours, that request up to 128 GPUs. The Helios trace consists of $\sim 250K$ jobs (Venus cluster) over 6 months, where jobs request multiple resource types, CPU and GPU. The Venus cluster trace is challenging due to a higher proportion of CPU jobs mixed with GPU jobs. Tab. 1 describes the job workload distribution for both traces. Jobs request up to 160 GPUs, and average 1.5 hours. For both traces, job runtime CDF is long-tailed; up to 80% of jobs run for less than an hour, while the longest jobs run for weeks.

For end-to-end results in simulation, we directly evaluate Starburst over *raw* traces, with no modifications to the original trace, to demonstrate Starburst’s applicability in real-world scenarios. We modify system load by varying the size of the

Load	Policy	Cost (\$)		JCT (hr.)		Cluster Util.	
		Real / Sim	Real / Sim	Real / Sim	Real / Sim	Real / Sim	Real / Sim
$s = 0.75$	No Wait	23.18 / 25.66		0.70 / 0.74		66.65 / 66.80	
	Constant Wait	21.33 / 21.33		0.72 / 0.77		70.39 / 71.95	
	Starburst (No-TE)	4.96 / 5.15		0.78 / 0.78		72.59 / 70.77	
	Starburst	4.73 / 5.35		0.74 / 0.76		70.73 / 71.38	
$s = 1.1$	No Wait	45.97 / 48.76		0.62 / 0.66		83.90 / 83.50	
	Constant Wait	33.89 / 37.06		0.67 / 0.72		83.30 / 84.25	
	Starburst (No-TE)	9.79 / 10.53		0.71 / 0.77		84.7 / 87.78	
	Starburst	8.96 / 7.13		0.66 / 0.71		85.20 / 87.69	

Table 3: **Physical cluster experiments.** This table measures total cloud costs, JCT, and cluster utilization from real world and simulation across system loads $s \in \{0.75, 1.1\}$.

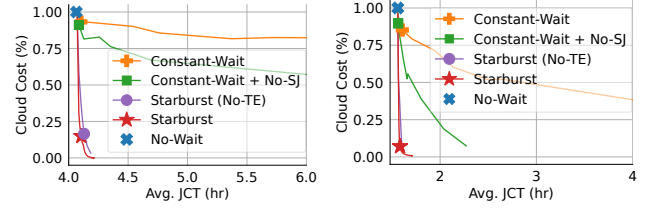
cluster, as timestamps in the raw traces determine when jobs arrive. Otherwise, we evaluate Starburst over *synthetic* traces, which use jobs from the raw trace but vary the arrival rate with respect to a Poisson distribution λ to modulate system load. 300K Jobs are randomly sampled from the original Philly or Helios trace, similar to prior work [53,55]. The synthetic trace is evaluated over a simulated 64 node cluster with 512 GPUs.

6.2 End-to-End Results on Physical Cluster

We evaluate Starburst and other baselines on our 32-GPU cluster at system loads of $s = 0.75, 1.1$, equivalent to 89 and 115 jobs arriving over a 3 hour period in our generated trace. We compare Starburst to No-Wait and Constant-Wait with respect to absolute costs, JCT, and cluster utilization in Tab. 3, totalling eight runs. In particular, for $s = 0.75$, Starburst reduces costs by **80%** over No-Wait, while increasing JCT by **5.7%**. Similarly, Starburst (No-TE) reduces costs by **78%** and increases JCT by **11.4%**. This indicates that Star-Wait can achieve similar performance even without a time estimator.

Simulator fidelity. We contrast real-world experiments with our simulator and observe minor differences in JCT ($\leq 8\%$), cluster utilization ($\leq 3\%$) and most costs ($\leq 10\%$). This demonstrates our simulator’s high fidelity for these metrics. However, we find up to 25% difference in costs for one baseline, Starburst with a time estimator. Despite these, Starburst consistently offers significant cost savings for organizations.

We find that these cost differences stem from time estimator errors that accrue over time. Conversely, when we submitted GPU sleep jobs that sleep for fixed times, simulation costs closely matched real-world costs. Our time estimator for AI workloads is not perfect. We run each model configuration offline for a single training epoch, collecting actual runtimes. We then estimate the total time for N epochs by multiplying the single epoch time with N , leading to a measured average time estimation error of 13% in real-world systems. Regardless of these inaccuracies, Starburst consistently outperforms No-Wait and Constant-Wait policies, underscoring its robustness to time estimation errors. This is further detailed in §A.6.



(a) Philly, $s = 0.83$.

(b) Helios, $s = 0.85$

Figure 8: **Pareto curves** represents the *synthetic* variants of **Philly** and **Helios** trace, over similar system loads. The markers on these curves symbolize points selected by the waiting budget B , with $P = 25$.

6.3 End-to-End Results in Simulation

We conduct a direct evaluation of Starburst using two *raw* production-level traces, Philly and Helios. These traces dictate job arrival, runtimes, and resource requests. Runtimes are used as time estimates for Starburst (Compute-Wait) and Constant-Wait + No-SJ. System load is varied by adjusting the cluster sizes between 16 and 144 nodes, with 8 GPUs per node.

Fig. 7a shows cost savings, JCT, and cluster utilization over the Helios trace. First, Starburst improves cost savings by **54.3-91.3%**, saving \$230-650K over the entire trace (200K jobs) for 48-80 node clusters, which corresponds to system load $s \in [0.6, 1)$. Similarly, Starburst with no time estimator (No-TE) improves cost savings by **37.8-80%**, performing within **16.5%** of Starburst. For system loads above cluster capacity ($s \geq 1$), cost savings diminish to 15% at $s = 1.5$. These observations match with established results in theoretical queuing theory [35,66]. Second, Starburst’s JCT increases by at most **5.3%** within system load $s \in [0.6, 1)$ and increases slower than Constant-Wait when $s \geq 2.5$; however, preempting short jobs (No-SJ) significantly decreases JCT. Lastly, No-Wait and Constant-Wait (with No-SJ) converge to 81% cluster utilization, but Starburst achieves up to **92.6%**.

The Philly results (Fig. 7b) tell a similar story. For system load $s \in [0.6, 1)$, Starburst improves cost savings by **30.2-69.8%**, saving \$134-225K over 100K jobs while increasing JCT by only **6.8%**. Starburst with no time estimator (No-TE) performs within **12%** of Starburst. As system load grows, Starburst achieves up to **99.2%** cluster utilization, increasing cluster utilization by 15% over other baselines. These results across multiple traces show the broad applicability of Starburst over many organizations with diverse workload distributions.

Pareto Curves. In Fig. 8, we assess Starburst over the *synthetic* Philly and Helios traces (300K jobs) at system load $s = 0.85$. We observe that Starburst substantially pulls the Pareto curve downwards³. Most efficiency improvements come from Out-of-Order scheduling. The markers indicate the positions of Starburst and other baselines on their own

³Constant-Wait operates on a First-Come-First-Serve (FCFS) basis, with Out-of-Order scheduling disabled. Therefore, its tradeoff curve appears flat, reaching zero costs at around 70 hours.

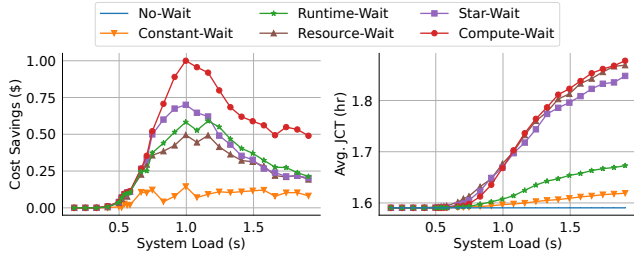
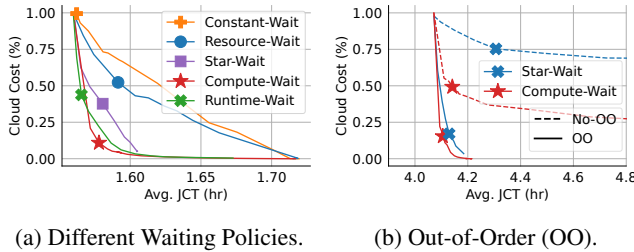


Figure 9: Ablating over different classes of waiting policies (with Out-of-order scheduling). Compute-Wait, followed by Star-Wait, achieves the highest cost savings, at the expense of higher JCT.



(a) Different Waiting Policies.

(b) Out-of-Order (OO).

Figure 10: Pareto curves representing different waiting policies (with OO scheduling) and scenarios including/excluding Out-of-Order scheduling. Markers on the curve mark points chosen by the waiting budget B , with $P = 25$.

Pareto curves, assuming they were allocated a $P = 25\%$ waiting budget. Notably, the Pareto curve of Starburst (No-TE) closely mirrors that of Starburst, showing that time-agnostic solutions can still attain good performance.

6.4 Analysis of Starburst's Design

This sections analyzes how each component of Starburst's design contributes to shifting the cost-JCT curve downwards. Then, we seek to understand how varying the waiting budget traverses the cost-JCT curve for different policies.

6.4.1 Impact of Waiting Policy

Fig. 9 compares the performance of various waiting policies influenced by job characteristics: Constant-Wait ($w(j) = C$), Runtime-Wait ($w(j) \propto t_j$), Resource-Wait ($w(j) \propto r_j$), Compute-Wait ($w(j) \propto r_j t_j$), and Star-Wait (Eq. 2), where job j requests r_j GPUs and runs for t_j . All policies have equal waiting budgets B , with $P = 25$, allocated across 300K jobs in the *synthetic* Helios trace. Compute-Wait attains highest cost savings, while increasing JCT by 6% at $s = 1$. However, this does not imply Compute-Wait is the best. Fig. 10a traces the cost-JCT curve for all policies, showing that Compute and Runtime-Wait have similar curves but differ in cost and JCT due to different waiting budget allocations. Notably, Star-Wait, without a time estimator, closely matches Compute-Wait's performance, effectively closing the performance gap.

6.4.2 Impact of Out-of-Order Scheduling

Fig. 11 shows that Out-of-Order (OO) scheduling benefits Compute-Wait and Star-Wait, with small cost improvements

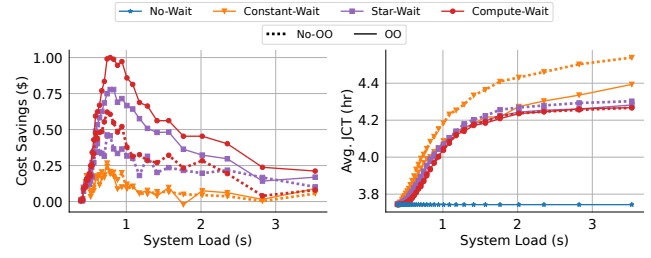
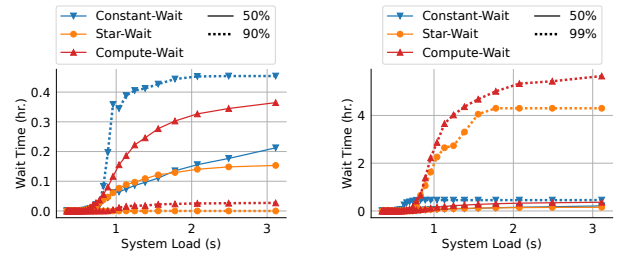
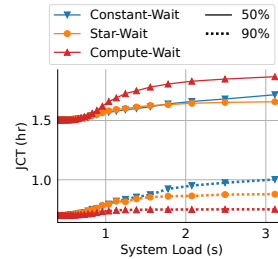


Figure 11: Adding Out-of-Order (OO) scheduling. OO scheduling complements Starburst's policies.



(a) 90th Percentile, Waiting

(b) 99th Percentile, Waiting



(c) 90th Percentile, JCT

(d) 99th Percentile, JCT

Figure 12: Understanding Starburst's impact on jobs' waiting and completion times (JCT). At the 90th percentile, Starburst achieves near zero waiting time for most jobs, outperforming other baselines. However, at the 99th percentile, Starburst increases the waiting time by up to 5 hours for the longest jobs, negatively affecting 99th percentile JCT.

for Constant-Wait. This is because OO scheduling effectively addresses head-of-line blocking, which occurs due to long wait times. In particular, Starburst assigns long waiting times to larger lobs, while Constant-Wait assigns smaller, equivalent wait times across all jobs. These findings are further validated when we evaluate OO scheduling over long and short waits in §A.5. Finally, in Figure 10b, OO scheduling significantly shifts the cost-JCT curve downward for Compute-Wait and Star-Wait, contributing to most of our performance gains.

6.4.3 Impact on Waiting and JCT

Figure 12 examines Starburst's impact on long-tail job waiting and completion times, particularly for larger jobs that are often subjected to longer waits. Fig. 12a reveals that the 90th percentile wait times for both versions of Starburst is close to 0 hrs, indicating small wait-time allocation for the large majority of jobs. Similarly, 90th percentile JCT

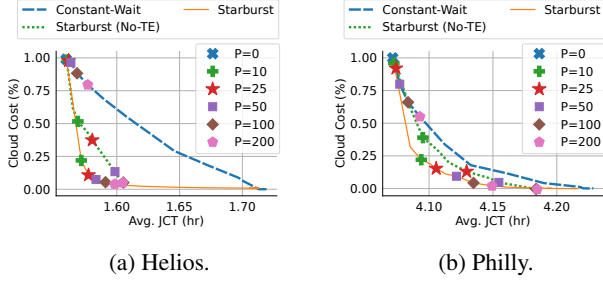


Figure 13: **Impact of waiting budget parameters ($P\%$) on the cost-JCT Pareto curve.** Starburst offers a more navigable waiting budget compared to Constant-Wait (with OO scheduling), frequently positioning the system near the Pareto curve’s knee.

(Fig. 12c) shows that Starburst’s completion times are smaller than that of other baselines. Conversely, the 99th percentile of wait times (Fig. 12b) for Starburst reaches around 5 hours. Hence, Fig. 12d shows that Starburst increases JCT for long jobs. However, this duration is relatively modest, given that 99th percentile of job runtime is around 40 hours for Philly and Helios traces.

6.4.4 Impact of Waiting Budget

This section examines how the waiting budget moves along the cost-JCT curve. Fig. 13 ablates different waiting budget parameters, P , for Philly and Helios trace and shows that JCT never exceeds $P\%$ of average job runtime (1.5 hr, Helios; 4 hr, Philly) for all data points. Notably, both versions of Starburst are sensitive to the waiting budget, with lower $P = 10, 25$ values positioned at the curve’s knee. In contrast, Constant-Wait’s waiting budget is not navigable and requires significantly higher P values to move along the curve.

6.5 Comparison to Optimal Scheduling

Computing the optimal allocation for each job, while assuming knowledge of future job arrivals, is unrealistic. Yet, we evaluate Starburst and the optimal Mixed Integer Linear Programming (MILP) formulation (§A.2) across 500 jobs on a 4-node cluster. This generates 500K variables to solve. The results of this comparison can be seen in Table 4. The MILP was solved using Gurobi [34]; we early-stopped the solving process at 8 hours. Job arrivals follow a Poisson process, with average job runtime at 1 hour. To keep waiting time constant across all baselines, Constant-Wait and Starburst’s hyperparameters (C, α) were tuned to achieve the same JCT across all 500 jobs, which determined the waiting budget for the MILP. We report the total cloud costs incurred and the relative cost savings (%) over No-Wait. Starburst consistently outperforms Constant-Wait, providing up to 50% additional cost savings. Furthermore, Starburst (Compute-Wait) falls within 5% of the optimal cloud cost under various system loads. This 5% discrepancy stems from bin-packing inefficiencies and the assumed knowledge of future job arrivals. These results indicate that Starburst’s performance is close to

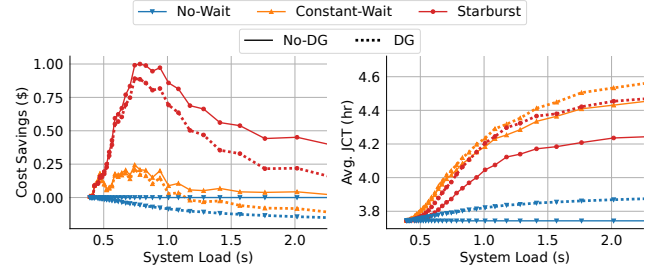


Figure 14: **Ablating data gravity (DG).** Data gravity increases cloud costs, JCT, and negatively impacts Starburst. However, Starburst still significantly outperforms other baselines.

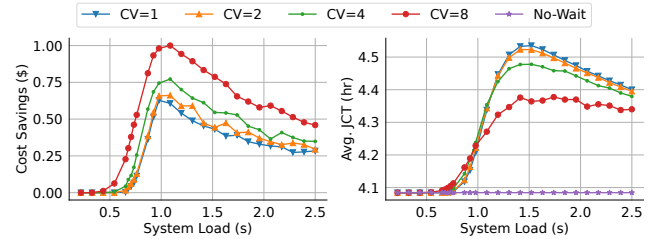


Figure 15: **Evaluating Starburst over increasingly bursty workloads (CV).** Increasing CV leads to increased cost savings and lower JCT.

the optimal solution, demonstrating that complex heuristics are unnecessary for cloud-enabled schedulers.

6.6 Evaluating Starburst’s Robustness

6.6.1 Data Gravity

This section examines the influence of data gravity and location on the performance of Starburst. We simulate a scenario where data resides locally, but jobs running in the cloud must egress their data dependencies from the local cluster. We assume a hypothetical worst-case scenario, where no data is cached in the cloud, and add a fixed delay of 15 minutes to all cloud-running jobs to represent data transfer time. Figure 14 demonstrates that data gravity increases cloud costs, leading to lower cost savings, and JCT across all baselines. Starburst is strongly impacted by data gravity, since it implicitly assigns a large proportion of small jobs to the cloud. Despite this, Starburst still shows significantly better cost and performance over other baselines.

6.6.2 Burstiness

This section evaluates Starburst’s robustness to bursty workloads using the Gamma distribution, $\Gamma(\alpha, \beta)$ [50, 74], to simulate job arrival rates. The increasing burstiness corresponds to a higher coefficient of variation (CV), $1/\sqrt{\alpha}$. Fig. 15 measures Starburst over the *synthetic* Philly trace with Gamma arrival rates. As the CV increases, we observe a **2x** increase in cost savings and 5% reduction in JCT. When CV increases, the intensity of job bursts and droughts becomes more pronounced. By allowing jobs to wait during bursty periods, computation

System Load	No-Wait	Constant-Wait	Starburst (No-TE)	Starburst	Optimal MILP	Avg. Waiting
$s = 0.75$	\$443.40 (-0%)	\$348.67 (-21.4%)	\$201.54 (-54.5%)	\$128.33 (-71.1%)	\$121.89 (-72.5%)	0.03 hr.
$s = 1$	\$760.68	\$705.55 (-7.25%)	\$607.31 (-20.1%)	\$551.68 (-27.5%)	\$526.80 (-30.7%)	0.05 hr.
$s = 1.5$	\$1055.14	\$912.86 (-13.5%)	\$850.54 (-19.4%)	\$817.35 (-22.5%)	\$781.24 (-25.96%)	0.12 hr.

Table 4: **Optimal MILP versus Starburst.** Starburst performs within 5% of the optimal cost.

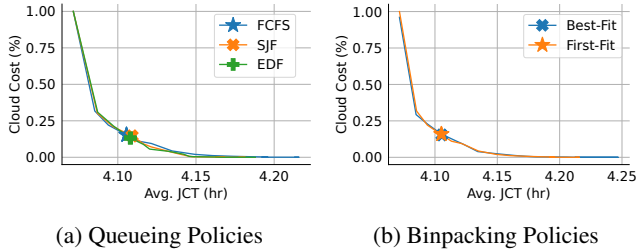


Figure 16: **Effect of queueing and binpacking policies.** Starburst exhibits invariance to the underlying scheduling policy. Markers symbolize points selected by the waiting budget, $P=25$.

is shifted to times when there are fewer incoming jobs (job droughts), leading to improved cluster utilization.

6.6.3 Starburst’s Invariance

We show that Starburst is invariant to different queueing and binpacking policies as depicted in Fig. 16. All policies with Starburst share nearly identical cost-JCT tradeoffs. This is due to Starburst’s Out-of-Order job scheduling mechanism, which simply loops through the queue. The choice of bin-packing algorithm also matters less in cloud-enabled scheduling, as *selecting* the right jobs to bin-pack is more important. This suggests developing the appropriate waiting policy is more impactful than the queueing or bin-packing policy.

7 Related Work

Job Scheduling. Cluster managers implement schedulers to orchestrate jobs and tasks over a fixed set of resources with a scheduling policy. First, a cluster scheduler decides when to run a job via a queueing policy, such as First Come First Serve (FCFS) [37, 64, 73], Shortest Job First (SJF) [67], Least Attained Service (LAS) [32], or general priority-based policies [46, 71]. Second, a scheduler determines the placement strategy (i.e. which node) via a bin-packing policy, such as First-Fit, Best-fit [71], Worst-fit [46, 65]. Bin-packing can also occur over single or multiple resource types [29, 30, 47, 53, 56]. Finally, schedulers may optimize for various objectives such as job completion time (JCT) [32, 53], throughput [55, 68], fairness [29, 72], and makespan [53, 55].

Cluster autoscaling. To orchestrate resources, cluster managers also implement cluster autoscalers to dynamically adjust cluster size via provisioned cloud instances. The default Kubernetes cluster autoscaler immediately scales cloud instances when tasks (i.e. pods) are pending due to insufficient cluster resources [46]. Various cloud services,

including managed instance groups [1, 8] and custom Kubernetes solutions [10, 12], scale by relying on rule-based mechanisms (e.g. CPU utilization $\geq 60\%$) and/or prediction-based methods to predict future bursts. Autoscalers may also optimize instance costs by right-sizing cloud instances [13] or managing multiple node pools [62]. Other studies [33, 52] focus on cloud bursting for on-premise jobs, emphasizing on cost-efficient data migration and meeting deadlines.

Orthogonally, there exists a large body of work on horizontal and vertical autoscaling, primarily focusing on resizing the number of tasks and task resource requirements per job. Works like Autopilot [59], Sinan [75], DS2 [41], and FIRM [57] either use direct feedback from workloads to maintain SLOs (e.g. latency or OOM rates) or use a model to predict resource needs. These approaches, which resubmit modified tasks to the scheduler and do not provision compute, are complementary to our work.

8 Conclusion

As organizations increasingly adopt hybrid cloud architectures, there is a strong incentive to design cloud-enabled schedulers that schedule jobs between the fixed cluster and cloud. Cloud-enabled schedulers are cost-aware and navigate a tradeoff between cloud costs and traditional scheduler objectives such as job completion time (JCT). To that end, we introduce Starburst, a cloud-enabled scheduler designed to maximize cluster utilization, which minimizes the cost of jobs sent to the cloud. Starburst introduces a waiting budget framework, which enables system admins to position their organization’s preferences on the cost-JCT tradeoff curve. We have implemented Starburst as a hybrid scheduler over Kubernetes, and show that it can save up to 91% in cloud costs while incurring a minor 5% increase in average JCT over real-life scenarios and multiple production-level traces. Through Starburst, we offer organizations a powerful tool for achieving an optimal balance in the complex cost-performance landscape of hybrid cloud environments.

Acknowledgement

We thank ATC reviewers and artifact evaluators for their valuable feedback and rigorous evaluation. We also thank Daniel Kang and Ameer Haj-ali for their insightful discussion while we were developing Starburst. This work is supported by gifts from Accenture, AMD, Anyscale, Google, IBM, Intel, Microsoft, Mohamed Bin Zayed University of Artificial Intelligence, Samsung SDS, SAP, Uber, and VMware.

References

- [1] AWS Autoscaling Groups. <https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html>.
- [2] AWS Kubernetes Service. <https://aws.amazon.com/eks/>.
- [3] Azure Hybrid Cloud. <https://azure.microsoft.com/en-us/solutions/hybrid-cloud-app>.
- [4] Cirrascale. <https://cirrascale.com/>.
- [5] CoreWeave. <https://coreweave.com/>.
- [6] FluidStack. <https://fluidstack.webflow.io/>.
- [7] GCP Hybrid Cloud. <https://cloud.google.com/solutions/hybrid-and-multicloud-application-platform>.
- [8] GCP Managed Instance Groups. <https://cloud.google.com/compute/docs/instance-groups/create-mig-with-basic-autoscaling>.
- [9] Goofys. <https://github.com/kahing/goofys>.
- [10] Google Kubernetes Engine. <https://cloud.google.com/kubernetes-engine/docs>.
- [11] IBM Hybrid Cloud. <https://www.ibm.com/hybrid-cloud>.
- [12] Karpenter. <https://karpenter.sh/>.
- [13] KubeCost. <https://www.kubecost.com>.
- [14] Kubernetes Cluster Autoscaler. <https://github.com/kubernetes/autoscaler>.
- [15] Lambda Cloud. <https://lambdalabs.com/>.
- [16] VMWare Hybrid Cloud. <https://www.vmware.com/cloud-solutions/hybrid-cloud.html>.
- [17] Pradeep Ambati, Noman Bashir, David Irwin, and Prashant Shenoy. Waiting game: Optimally provisioning fixed resources for cloud-enabled schedulers. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–14, 2020.
- [18] Pradeep Ambati, Noman Bashir, David Irwin, and Prashant Shenoy. Good things come to those who wait: Optimizing job waiting in the cloud. In *Proceedings of the ACM Symposium on Cloud Computing, SoCC '21*, page 229–242, New York, NY, USA, 2021. Association for Computing Machinery.
- [19] Nikhil Bansal, José R. Correa, Claire Kenyon, and Maxim Sviridenko. Bin packing in multiple dimensions: Inapproximability results and approximation schemes. *Math. Oper. Res.*, 31(1):31–49, 2006.
- [20] Christian Blum and Verena Schmid. Solving the 2d bin packing problem by means of a hybrid evolutionary algorithm. In Vassil N. Alexandrov, Michael Lees, Valeria V. Krzhizhanovskaya, Jack J. Dongarra, and Peter M. A. Sloot, editors, *Proceedings of the International Conference on Computational Science, ICCS 2013, Barcelona, Spain, 5-7 June, 2013*, volume 18 of *Procedia Computer Science*, pages 899–908. Elsevier, 2013.
- [21] Sébastien Bocquet. Queueing theory with reneging. 2005.
- [22] Nestor Cid-Garcia and Yasmin Rios. Positions and covering: A two-stage methodology to obtain optimal solutions for the 2d-bin packing problem. *PLOS ONE*, 15:e0229358, 04 2020.
- [23] Michael B. Cohen, Yin Tat Lee, and Zhao Song. Solving linear programs in the current matrix multiplication time, 2020.
- [24] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: a flexible data processing tool. *Communications of the ACM*, 53(1):72–77, 2010.
- [25] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [26] Stephen Merity et al. Wikitext-103. 2016.
- [27] Stephen Merity et al. Wikitext-2. 2016.
- [28] Armando Fox, Rean Griffith, Anthony Joseph, Randy Katz, Andrew Konwinski, Gunho Lee, David Patterson, Ariel Rabkin, Ion Stoica, et al. Above the clouds: A berkeley view of cloud computing. *Dept. Electrical Eng. and Comput. Sciences, University of California, Berkeley, Rep. UCB/EECS*, 28(13):2009, 2009.
- [29] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation, NSDI'11*, page 323–336, USA, 2011. USENIX Association.
- [30] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. Multi-resource packing for cluster schedulers. In *Proceedings of the 2014 ACM Conference on SIGCOMM, SIGCOMM '14*, page 455–466, New York, NY, USA, 2014. Association for Computing Machinery.

- [31] Donald Gross, John F. Shortle, James M. Thompson, and Carl M. Harris. *Fundamentals of Queueing Theory*. Wiley-Interscience, USA, 4th edition, 2008.
- [32] Juncheng Gu, Mosharaf Chowdhury, Kang G. Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. Tiresias: A gpu cluster manager for distributed deep learning. In *Proceedings of the 16th USENIX Conference on Networked Systems Design and Implementation*, NSDI'19, page 485–500, USA, 2019. USENIX Association.
- [33] Tian Guo, Upendra Sharma, Timothy Wood, Sambit Sahu, and Prashant Shenoy. Seagull: Intelligent cloud bursting for enterprise applications. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pages 361–366, Boston, MA, June 2012. USENIX Association.
- [34] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2023.
- [35] Frank A. Haight. Queueing with reneging. *Metrika*, 2(1):186–197, 1959.
- [36] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.
- [37] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy Katz, Scott Shenker, and Ion Stoica. Mesos: A platform for fine-grained resource sharing in the data center. In *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation*, NSDI'11, page 295–308, USA, 2011. USENIX Association.
- [38] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for mobilenetv3, 2019.
- [39] Qinghao Hu, Peng Sun, Shengen Yan, Yonggang Wen, and Tianwei Zhang. Characterization and prediction of deep learning workloads in large-scale gpu datacenters. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '21, New York, NY, USA, 2021. Association for Computing Machinery.
- [40] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, unjie Qian, Wencong Xiao, and Fan Yang. Analysis of large-scale multi-tenant gpu clusters for dnn training workloads. In *Proceedings of the 2019 USENIX Conference on Usenix Annual Technical Conference*, USENIX ATC '19, page 947–960, USA, 2019. USENIX Association.
- [41] Vasiliki Kalavri, John Liagouris, Moritz Hoffmann, Desislava Dimitrova, Matthew Forshaw, and Timothy Roscoe. Three steps is all you need: fast, accurate, automatic scaling decisions for distributed streaming dataflows. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 783–798, 2018.
- [42] Sergey Karayev and Charles Frye. Cloud GPUs. <https://fullstackdeeplearning.com/cloud-gpus/>, 2023. [Online; accessed 01-Feb-2023].
- [43] Chris Kleban and Ari Liberman. Expanding our GPU portfolio with NVIDIA Tesla V100. <https://tinyurl.com/yyfkr7z8>, 2018. [Online; accessed 30-April-2018].
- [44] Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. Cifar-10 (canadian institute for advanced research).
- [45] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
- [46] Kubernetes Manual. <https://kubernetes.io/docs/home/>, 2017. [Online; accessed 04-Dec-2017].
- [47] Tan N. Le, Xiao Sun, Mosharaf Chowdhury, and Zhenhua Liu. Allox: Compute allocation in hybrid clusters. In *Proceedings of the Fifteenth European Conference on Computer Systems*, EuroSys '20, New York, NY, USA, 2020. Association for Computing Machinery.
- [48] Jérôme Lelong, Valentin Reis, and Denis Trystram. Tuning easy-backfilling queues. In Dalibor Klusáček, Walfredo Cirne, and Narayan Desai, editors, *Job Scheduling Strategies for Parallel Processing - 21st International Workshop, JSSPP 2017, Orlando, FL, USA, June 2, 2017, Revised Selected Papers*, volume 10773 of *Lecture Notes in Computer Science*, pages 43–61. Springer, 2017.
- [49] Y. Li, T. Li, T. Kahveci, and J. Fortes. Workload characterization of bioinformatics applications. In *13th IEEE International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems*, pages 15–22, 2005.
- [50] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Alpaserve: Statistical multiplexing with model parallelism for deep learning serving, 2023.
- [51] IBM Spectrum LSF. <https://www.ibm.com/docs/en/spectrum-lsf/10.1.0>, 2017. [Online; accessed 17-Jan-2023].

- [52] Ming Mao and Marty Humphrey. Auto-scaling to minimize cost and meet application deadlines in cloud workflows. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '11, New York, NY, USA, 2011. Association for Computing Machinery.
- [53] Jayashree Mohan, Amar Phanishayee, Janardhan Kulkarni, and Vijay Chidambaram. Synergy: Resource sensitive DNN scheduling in multi-tenant clusters. *CoRR*, abs/2110.06073, 2021.
- [54] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. *CoRR*, abs/1712.05889, 2017.
- [55] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-Aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 481–498. USENIX Association, November 2020.
- [56] Yanghua Peng, Yixin Bao, Yangrui Chen, Chuan Wu, and Chuanxiong Guo. Optimus: An efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth EuroSys Conference*, EuroSys '18, New York, NY, USA, 2018. Association for Computing Machinery.
- [57] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. FIRM: An intelligent fine-grained resource management framework for SLO-Oriented microservices. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 805–825. USENIX Association, November 2020.
- [58] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [59] Krzysztof Rządca, Paweł Findeisen, Jacek Swiderski, Przemysław Zych, Przemysław Broniek, Jarek Kusmerek, Paweł Nowak, Beata Strack, Piotr Witusowski, Steven Hand, et al. Autopilot: workload autoscaling at google. In *Proceedings of the Fifteenth European Conference on Computer Systems*, pages 1–16, 2020.
- [60] Rico Sennrich, Barry Haddow, and Alexandra Birch. Edinburgh neural machine translation systems for wmt 16, 2016.
- [61] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition, 2015.
- [62] Mulugeta Ayalew Tamiru, Johan Tordsson, Erik Elmroth, and Guillaume Pierre. An experimental evaluation of the kubernetes cluster autoscaler in the cloud. In *2020 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, pages 17–24, 2020.
- [63] Mingxing Tan and Quoc V. Le. Efficientnetv2: Smaller models and faster training, 2021.
- [64] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, Bikas Saha, Carlo Curino, Owen O'Malley, Sanjay Radia, Benjamin Reed, and Eric Baldeschwieler. Apache hadoop yarn: Yet another resource negotiator. In *Proceedings of the 4th Annual Symposium on Cloud Computing*, SOCC '13, New York, NY, USA, 2013. Association for Computing Machinery.
- [65] Abhishek Verma, Luis Pedrosa, Madhukar R. Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, Bordeaux, France, 2015.
- [66] Qiangqiang Wang and Bin Zhang. Analysis of a busy period queuing system with balking, reneging and motivating. *Applied Mathematical Modelling*, 64:480–488, 2018.
- [67] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. MLaaS in the wild: Workload analysis and scheduling in Large-Scale heterogeneous GPU clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 945–960, Renton, WA, April 2022. USENIX Association.
- [68] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. Gandiva: Introspective cluster scheduling for deep learning. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation*, OSDI'18, page 595–610, USA, 2018. USENIX Association.
- [69] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks, 2017.
- [70] Zongheng Yang, Zhanghao Wu, Michael Luo, Wei-Lin Chiang, Romil Bhardwaj, Woosuk Kwon, Siyuan

Zhuang, Frank Sifei Luan, Gautam Mittal, Scott Shenker, and Ion Stoica. SkyPilot: An intercloud broker for sky computing. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 437–455, Boston, MA, April 2023. USENIX Association.

- [71] Andy B. Yoo, Morris A. Jette, and Mark Grondona. Slurm: Simple linux utility for resource management. In Dror Feitelson, Larry Rudolph, and Uwe Schwiegelshohn, editors, *Job Scheduling Strategies for Parallel Processing*, pages 44–60, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.
- [72] Matei Zaharia, Dhruba Borthakur, Joydeep Sen Sarma, Khaled Elmeleegy, Scott Shenker, and Ion Stoica. Delay scheduling: A simple technique for achieving locality and fairness in cluster scheduling. In *Proceedings of the 5th European Conference on Computer Systems, EuroSys '10*, page 265–278, New York, NY, USA, 2010. Association for Computing Machinery.
- [73] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster computing with working sets. In *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing, HotCloud'10*, page 10, USA, 2010. USENIX Association.
- [74] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. SHEPHERD: Serving DNNs in the wild. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 787–808, Boston, MA, April 2023. USENIX Association.
- [75] Yanqi Zhang, Weizhe Hua, Zhuangzhuang Zhou, G Edward Suh, and Christina Delimitrou. Sinan: MI-based and qos-aware resource management for cloud microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 167–181, 2021.

A Appendix

A.1 Artifact Appendix

Abstract

Our artifact provides a comprehensive tutorial to validate and replicate the results in the Starburst paper. Our artifact offers detailed instructions for setting up the environment, running experiments for both simulation and real life system, and plotting the graphs presented in the Starburst paper. Our artifact is hosted on GitHub at <https://github.com/michaelzhiluo/starburst>, where we continuously update code and maintain reproducible instructions for users in a `README.md` file.

Scope

This artifact is designed to validate the claims made in the Starburst paper and provides all necessary tools for replicating the experiments. It helps users set up the required Python environment efficiently. For the simulator experiments, the objective is to closely replicate the results presented in the figures and tables of the paper. For the real-life experiments, our goal is to validate the Starburst’s claims, that it can save 54-91% cloud costs and that the real-life deployment of Starburst closely aligns with the simulator’s results.

Contents

Our artifact offers comprehensive guidance for users on installing the necessary components and conducting experiments as detailed in the Starburst paper. It is divided into three main sections: Setup, Simulation Experiments, and Real-life Experiments.

In the Setup section, users will find detailed instructions for installing the required Python dependencies, downloading workload traces for simulation, and setting up Gurobi. These traces are utilized in all simulation experiments.

The Simulation Experiments section provides step-by-step instructions for replicating the primary experiments presented in our paper—such as the Philly and Helios results, ablations over Starburst’s components, and comparisons with the optimal formulation. These experiments may take up to 3 hours for each to complete. If needed, we also include output logs from these experiments in a public bucket. Finally, we provide plotting notebooks to help users reproduce the graphs presented in the Starburst paper.

The Real-life Experiments section outlines how to deploy Starburst as a higher-level scheduler, integrate Chakra to manage best-fit binpacking on the underlying Kubernetes cluster, and create a job submission service that continuously submits jobs to Starburst while tracking critical job statistics, such as runtime, waiting time, and start time.

Hosting

Our artifact is available on GitHub, hosted under the most recent commit on the main branch at: <https://github.com/michaelzhiluo/starburst>.

[//github.com/michaelzhiluo/starburst](https://github.com/michaelzhiluo/starburst). We also maintain a public bucket that contains all logs from both simulator and real-life experiments. We will continue to update the GitHub repository and maintain a `README.md` file, which contains clear and easy-to-follow instructions about our artifacts.

Requirements

The simulator is designed to run parallel simulations efficiently on multicore systems; therefore, a device with multiple cores is recommended to complete our runs in reasonable time. For real-life deployments, Starburst has been developed and rigorously tested on a Kubernetes cluster acting as the local cluster. It utilizes SkyPilot for provisioning VMs across multiple cloud providers for cloud-based jobs. For users bounded by cost or without access to cloud resources, we offer an alternative to log cloud-directed jobs locally to a file.

A.2 Full Formulation

Since cloud-enabled schedulers balance cost and JCT, we formulate the scheduling objective as a constrained optimization problem. The goal is to maximize cluster utilization, which is akin to minimizing cloud costs, while upper-bounding job completion time (JCT).

We express cloud-enabled scheduling as a mixed integer linear program (MILP). The MILP solver optimizes over 1) bin-packing, which defines placement of jobs, and 2) waiting time allocation, which distributes waiting time to J jobs from a waiting budget B . This formulation is a constrained-form of multi-dimensional bin packing [20,22], which is NP-hard [19]. The MILP formulation presumes *a priori* knowledge of job arrivals and focuses on a single resource type, such as GPUs, in a homogeneous cluster. Nonetheless, it can be adapted for multiple resources and heterogeneous nodes.

We assume a cluster comprising N nodes, each containing G GPUs, and consider J jobs. Each job j arrives at time a_j , runs for t_j , and requests n_j nodes, each with r_j GPUs. The variables for the MILP are as follows:

- $S \in \{0, 1\}^{N \times J}$ - 1 if job j is in node n , 0 otherwise.
- $c \in \{0, 1\}^J$ - 1 if job j runs on cluster, 0 if cloud.
- $w \in \mathbb{R}_{\geq 0}^J$ - Job j ’s allocated wait time.
- $(x \in \mathbb{R}_{\geq 0}^J, y \in \mathbb{Z}_{\geq 0}^J)$ - Job j starts at time x_j and occupies GPUs $\{y_j, \dots, y_j + r_j\}$ on n_j nodes.
- $(L \in \{0, 1\}^{J \times J}, B \in \{0, 1\}^{J \times J})$ - 1 if job i is below and/or to the left of job j , 0 otherwise.

Our objective function is to maximize cluster utilization. We formulate it using the above MILP variables. Z is an arbitrarily large number.

$$\text{Maximize} \quad \sum_n \sum_j S_{nj} \cdot r_j \cdot t_j \quad (3)$$

Next we enforce the following constraints such that the final MILP solution performs bin-packing and wait allocation:

$$x_j \geq a_j; x_j \leq a_j + w_j, \forall j \quad (4)$$

$$y_j \geq 0; y_j \leq G - r_j, \forall j \quad (5)$$

$$\sum_n S_{nj} = c_j \cdot n_j, \forall j \quad (6)$$

$$x_i - x_j + Z \cdot L_{ij} \leq Z - w_i, \forall i, j \quad (7)$$

$$y_i - y_j + G \cdot B_{ij} \leq G - r_i, \forall i, j \quad (8)$$

$$L_{ij} + L_{ji} + B_{ij} + B_{ji} + (2 - S_{ni} - S_{nj}) \geq 1, \forall i, j, n; i < j \quad (9)$$

$$\sum_j w_j \leq B \quad (10)$$

Constraints (4), (5) ensure that jobs do not exceed their allocated wait times and fit within GPU indexes $\{0, 1, \dots, G-1\}$ on every node. Constraint (6) ensures that, if job j runs on cluster, it will run on n_j nodes, and, if j is sent to the cloud, it runs on 0 cluster nodes. Constraints (7), (8), and (9) enforce that no overlaps occur between two jobs on the same node. Finally, Constraint (10) guarantees that the accumulated wait across all jobs does not exceed more than the waiting budget B , hence enforcing an upper bound on JCT. We evaluate Starburst against the optimal formulation above in §6.5.

Takeaways from Optimal. While the MILP formulation selects the right jobs to perform optimal bin-packing, there are several challenges that prevent practical deployment. First, our multi-dimensional bin-packing formulation is computationally expensive. The total variables to ensure that the jobs do not intersect spatially and temporally (Eqn. 7, 8, 9) scales quadratically with total jobs J . Even worse, the time to solve increases exponentially when both cluster size and jobs increase [23]. Next, the MILP formulation assumes knowledge of job arrivals, which is unrepresentative of real-life workloads. The MILP could be applied periodically as an online planner, but this trades off with compute intensity.

A.3 Star-Wait and the Waiting Budget

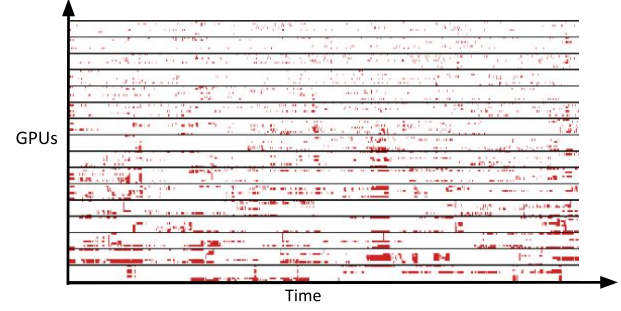
This section derives a simple approximation to estimate Star-Wait's hyperparameter β . From before, the waiting budget, B , is equal to the total waiting time allocated across all jobs: $B = PTN/100$, where $P\%$ is the admin's desired increased in runtime, T is the average runtime, and N is the total number of jobs.

First, we first compute Resource-Wait's parameter, β_r , where jobs j request r_j resources:

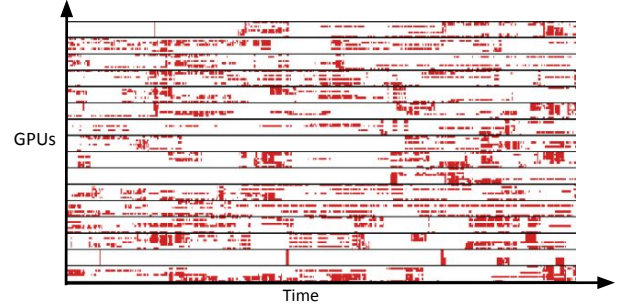
$$\beta_r = \frac{B}{\sum_j r_j} = \frac{PTN}{(\sum_j r_j) \cdot 100} \approx \frac{PT}{100 \cdot \mathbb{E}[r]} \quad (11)$$

Resource-Wait depends on the average job runtime, T , and the average job size, $\mathbb{E}[r]$. These can be tracked as more jobs are submitted through Starburst.

Star-Wait's policy employs No-Wait and Resource-Wait for non-preempted and preempted jobs respectively. Preemption places a *filter* for jobs that are assigned wait times. This can be represented as an indicator function $\mathbb{1}_{\text{preempt}}$, which



(a) No-Wait ($C=0$).



(b) Constant-Wait ($C=4$).

Figure 17: **Cluster execution traces illustrating running jobs (white) and cluster under-utilization (red) over a 16 GPU time slice and high system load ($s = 1.5$).** (a) The No-Wait policy primarily schedules small jobs, leading to small pockets of cluster under-utilization. (b) Constant-Wait accommodates larger jobs, resulting in larger and longer chunks of cluster under-utilization due to head-of-line blocking.

returns 1 if a job is preempted and 0 otherwise. The indicator function depends on two probabilities. The first probability, p_1 , denotes the fraction of jobs that initially spill over to the cloud via the No-Wait policy. From queuing theory, p_1 is equivalent to Erlang's loss formula [21, 35]. The second probability, p_2 , denotes the fraction of jobs in the cloud that exceed $T_{\max}$. Assuming the preemption threshold $T_{\max}$ is small, Star-Wait's parameter, β , can then be computed as:

$$\beta = \frac{B}{\sum_j \mathbb{1}_{\text{preempt}} \cdot r_j} \approx \frac{PT}{100 \cdot \mathbb{E}[\mathbb{1}_{\text{preempt}}] \cdot \mathbb{E}[r]} \approx \frac{\beta_r}{p_1 \cdot p_2} \quad (12)$$

To avoid computing p_1 and p_2 , we propose an additional simplification to approximate β . Star-Wait's parameter is instead scaled by a fixed value X over Resource-Wait's parameter: $\beta \approx X \cdot \beta_r$. We set $X = 3$ for all our experiments for Starburst (No-TE). The guarantee that the waiting budget never exceeds $P\%$ of average job runtime holds true as long as the average system load, s , is upper bounded. To show why, we illustrate an example from the Philly trace [40]. The fraction of jobs that exceed $T_{\max}$ (15 minutes) is $p_2 = \frac{1}{2}$. This means that $p_1 \leq \frac{1}{X \cdot p_2} = \frac{2}{3}$, or at most two-thirds of jobs must spillover to the cloud. This corresponds to a system load $s \leq 3$. If system load exceeds its bounds, the waiting budget will assign waiting time to too many jobs.

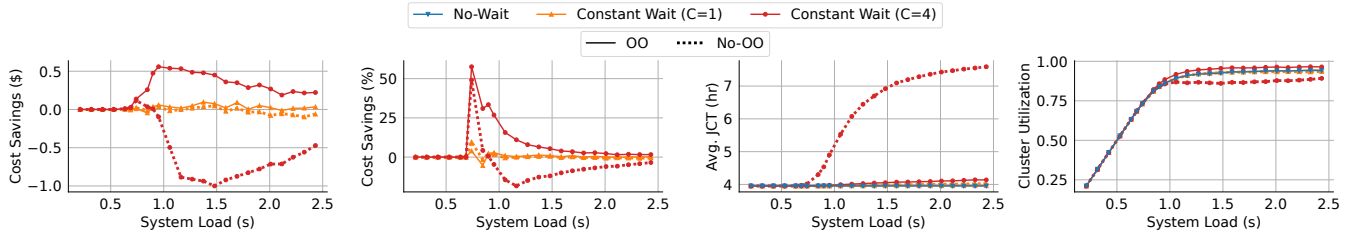


Figure 18: **Absolute and Relative Cost Savings (\$, %) over No-Wait, JCT, and Cluster Utilization for Constant Wait with Out-of-Order (OO) scheduling ($C = 1, 4$) over Microsoft Philly production trace [40].** Positive cost savings indicate lower cloud costs than No-Wait. OO scheduling does not impact short waits ($C=1$), but presents significant improvements to cost savings and JCT for long waits ($C=4$).

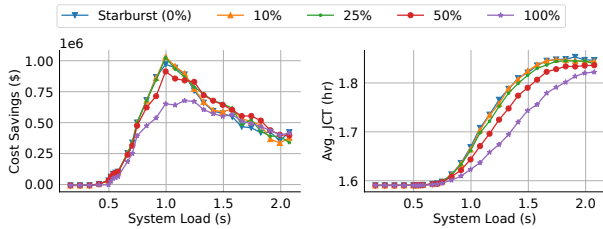


Figure 19: **Time-Estimator errors.** Starburst is robust to $e = 50\%$ error in job runtime inaccuracies, modeled as $\hat{t}_j = \mathcal{N}(t_j, e * t_j)$

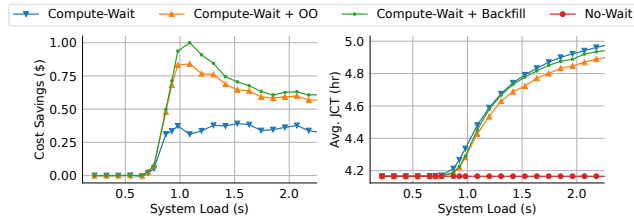


Figure 20: **Out-of-Order versus Backfill scheduling.** Out-of-Order achieves similar cost savings and JCT to that of backfill.

A.4 Gantt Chart Visualization

Fig. 17 shows the cluster execution traces for No-Wait and Constant-Wait, where horizontal and vertical axis correspond to time and GPU resources. Red and white space corresponds to cluster under-utilization and running jobs. We observe job starvation in Fig. 17a for the No-Wait policy. Since jobs do not wait, large jobs are unlikely to find sufficient resources in the cluster. Hence, only small jobs run on the cluster, which leads to many small, narrow pockets of under-utilization. Conversely, we observe head-of-line blocking in Fig. 17b for the Constant-Wait policy ($C = 4$). Since jobs are assigned four hours of wait, large jobs block the queue, which forcibly idles available resources. This creates larger and longer chunks of cluster under-utilization. Overall, when system load is large ($s > 1$), scheduling large jobs incurs a larger opportunity cost than that of scheduling only small jobs; hence, No-Wait achieves high cluster utilization than Constant-Wait.

A.5 Short vs. Long Waits with OO Scheduling

Fig. 18 illustrates the effects of Out-of-Order (OO) scheduling on both short and long waits (Constant-Wait, $C \in \{1, 4\}$), focusing on cost savings compared to No-Wait, JCT, and cluster utilization. For short waits, there is no visible cost improvements. However, for long waits, OO scheduling significantly improves cost and JCT. OO scheduling eliminates under-utilization stemming from head-of-line blocking, which grows with system load, s . Hence, the cost savings for long waits increase from -20% to 15% at $s = 1.2$. Consequently, JCT is reduced, as smaller jobs are no longer delayed by larger jobs.

A.6 Time Estimator Errors

Starburst's Compute Wait assumes predictable job runtimes, such as recurring tasks [67] or finetuning DNN models over several epochs of data. We aim to assess the resilience of Starburst in the face of inaccurate time predictions. As Fig. 19 illustrates, Starburst's performance, with noise introduced to job runtimes t_j , $\hat{t}_j = \mathcal{N}(t_j, e * t_j)$, remains robust up to a $e = 50\%$ error margin. At 100% error, peak savings drop by up to 35%, offset by a small JCT reduction. These results indicate that Compute-Wait is robust to large prediction errors stemming from faulty or inaccurate models.

A.7 Starburst vs. Backfill Scheduling

Fig. 20 ablates the performance of out-of-order scheduling to backfill, a commonly-used HPC scheduling algorithm [51, 71], that performs explicit reservations for large jobs and requires precise time estimates. Both baselines use Compute-Wait for the waiting policy and the experiment is conducted over the *synthetic* Philly trace [40]. At system load $s = 1$, backfill scheduling achieves a peak of 10% more cost savings than OO scheduling at the cost of a modest 2% increase in JCT. These results highlight OO scheduling as an effective and implicit approximation to backfill.

A.8 Impact of JCT Offloading Methods

Fig. 21 illustrates the impact of job offloading heuristics on JCT using the *synthetic* Helios trace. Immediately sending CPU-only jobs to the cloud prevents CPU jobs from blocking GPU jobs, decreasing JCT by 3%. Limiting queue size ($Q_{\max} = 10$) increases the proportion of balking jobs with

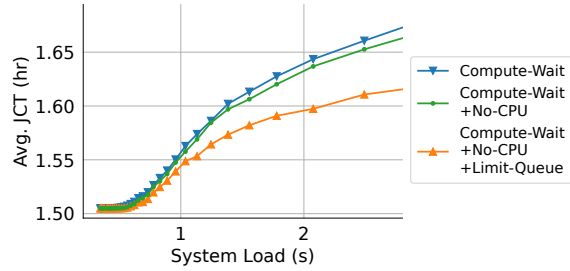


Figure 21: **Impact of job-offloading policies:** No-CPU & Limit-Queue.

GPU	Cost	GPU:vCPU
H100	\$11.09/hr	220:1
A100	\$3.07/hr	60:1
V100	\$2.50/hr	50:1
P100	\$1.46/hr	30:1
K80	\$0.45/hr	9:1
T4	\$0.18/hr	3.6:1

Table 5: **Approximate values and ratios for GPU to vCPU pricing for major cloud providers during Jan 2024.** Most GPU(s) types are at least an order of magnitude more expensive than vCPU(s).

zero wait times, reducing JCT by up to 12%.

A.9 GPU-to-CPU Pricing

We evaluate the cost ratio of GPU to CPU across heterogeneous GPUs. Tab. 5 shows the cost ratio of GPU to CPU jobs for major cloud providers, pulled from various catalogs [42, 43, 70]. The data shows that GPU usage is significantly more expensive than CPU usage, with GPU prices being at least ten times higher than the standard vCPU price of \$0.05/hr. Notably, the H100 GPU is priced particularly high, a trend attributed to an artificial GPU shortage at the time of this writing. It’s important to note that this high cost differential might be a temporary situation. Additionally, smaller GPU cloud services are emerging, offering GPUs at substantially lower prices compared to major cloud providers [4–6, 15].